

IMPLEMENTASI FACE RECOGNITION DAN PLATE DETECTION

Menggunakan OpenCV pada Google Colab

Eolia Shalbillah - 2318039

FACE RECOGNITION

Face Recognition digunakan untuk membantu komputer mengenali keberadaan wajah pada sebuah gambar. Sistem akan mencari pola tertentu pada area wajah seperti mata, hidung, dan bentuk kepala sehingga objek dapat dibedakan dari background. Pada percobaan ini, proses deteksi dilakukan menggunakan OpenCV dan Haar Cascade untuk membaca wajah dari beberapa citra yang telah disiapkan.

Teknologi ini banyak digunakan pada:

- sistem keamanan
- absensi digital
- smartphone

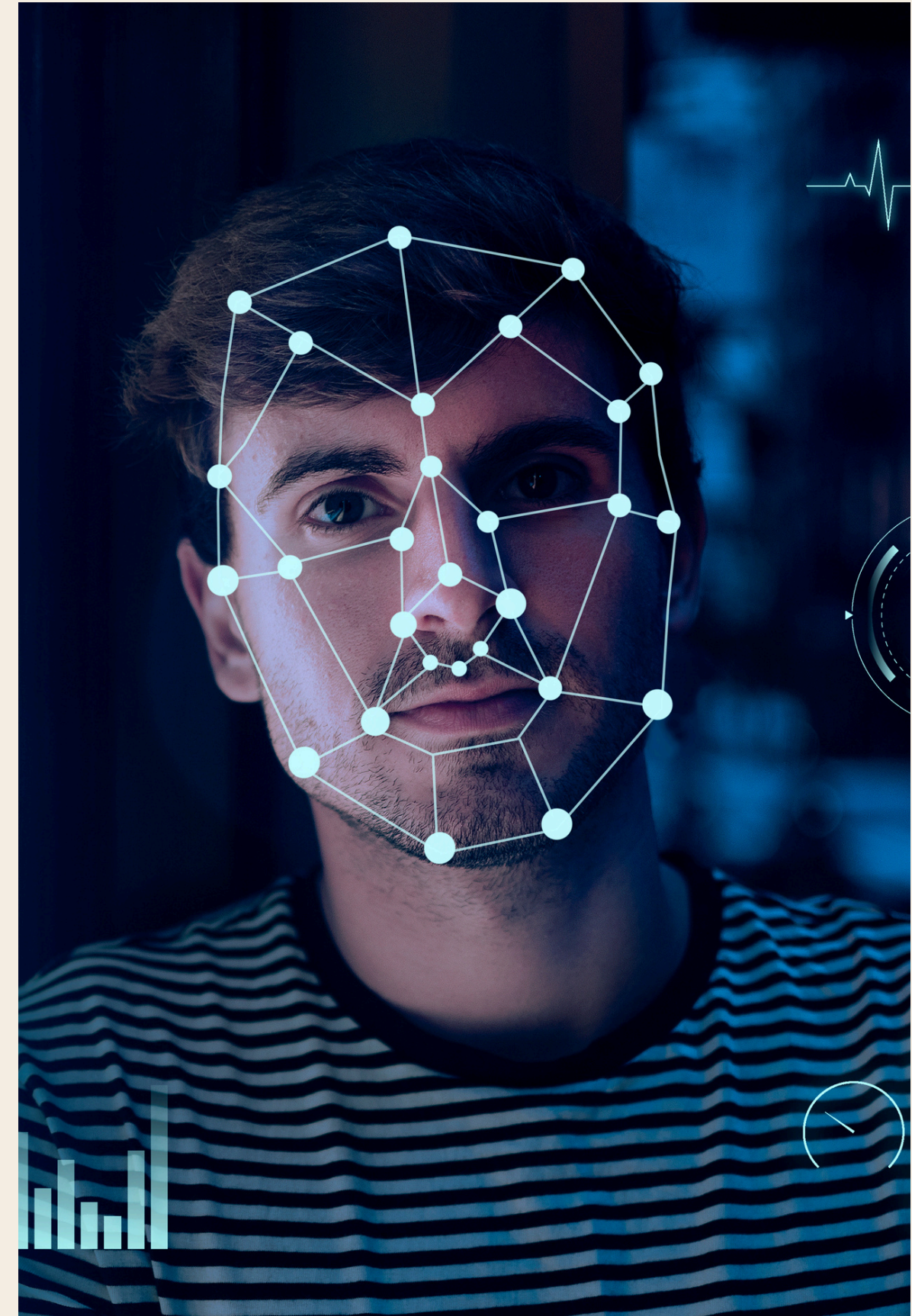


PLATE DETECTION

Plate Detection digunakan untuk mencari posisi plat kendaraan pada citra. Setelah area plat ditemukan, sistem dapat memberikan tanda pada plat dan melakukan pembacaan karakter menggunakan OCR (membaca teks dari citra).

Hasil deteksi dipengaruhi oleh:

- pencahayaan gambar
- posisi kendaraan
- ukuran plat
- kualitas citra



HAAR CASCADE

Haar Cascade merupakan metode deteksi objek yang bekerja dengan membaca pola kontras pada gambar. Sistem akan melakukan scanning pada banyak area kecil untuk mencari ciri objek tertentu.

Pada implementasi ini, Haar Cascade digunakan karena proses deteksinya cukup cepat dan mudah diterapkan pada OpenCV.

Keunggulan:

- cepat
- ringan
- cocok untuk real-time detection



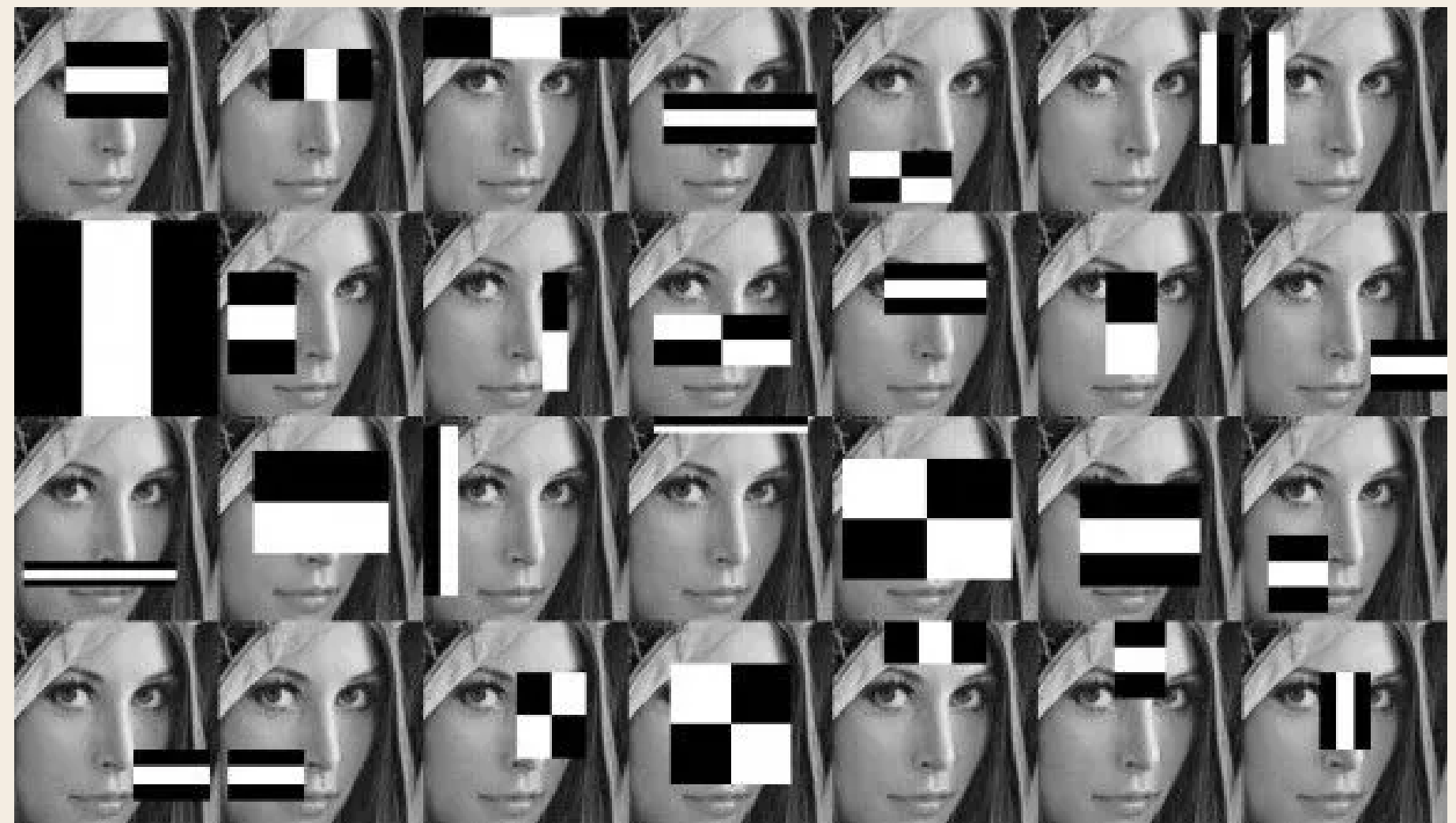
PERHITUNGAN HAAR CASCADE (MENCARI POLA)

Haar Cascade menggunakan beberapa proses perhitungan untuk membantu sistem mendeteksi objek pada citra dengan lebih cepat dan efisien.

Tahapan utama:

- Haar Feature
- Integral Image
- AdaBoost
- Cascade Classifier

Keempat proses tersebut bekerja untuk mengenali pola objek berdasarkan perbedaan intensitas warna pada gambar.



PROSES HAAR FEATURE

Haar Feature digunakan untuk membandingkan area terang dan area gelap pada citra.

Contoh:

- Area terang = 90
- Area gelap = 40

Maka:

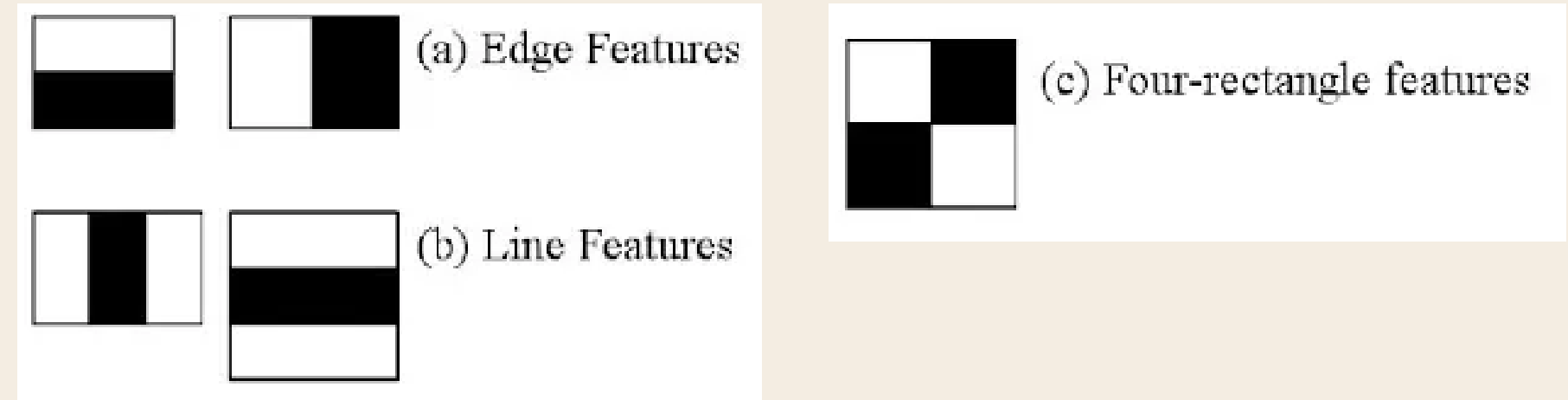
- $90 - 40 = 50$

Nilai selisih tersebut digunakan sistem untuk mengenali pola wajah atau objek tertentu pada gambar.

Cara Kerja:

- area putih $\rightarrow$ nilai terang
- area hitam $\rightarrow$ nilai gelap
- sistem menghitung selisih kedua area

Semakin jelas perbedaan nilainya, semakin mudah objek dikenali.



Kotak-kotak hitam dan putih yang ditempelkan di atas wajah atau objek.

- Cara Kerjanya: Komputer membandingkan intensitas cahaya. Contohnya, area mata biasanya lebih gelap (kotak hitam) daripada area tulang pipi (kotak putih).
- Nilai Fitur: Dihitung dengan rumus sederhana: (Jumlah Piksel di Area Putih) - (Jumlah Piksel di Area Hitam).
- Logikanya: Jika selisihnya besar dan sesuai pola, komputer akan berbisik ini objek sesuatu (hidung/mulut/mata/dll)

PROSES INTEGRAL IMAGE

Integral Image digunakan untuk mempercepat proses perhitungan piksel pada citra.

Contoh matriks awal:

$$\begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix}$$

Perhitungan integral image:

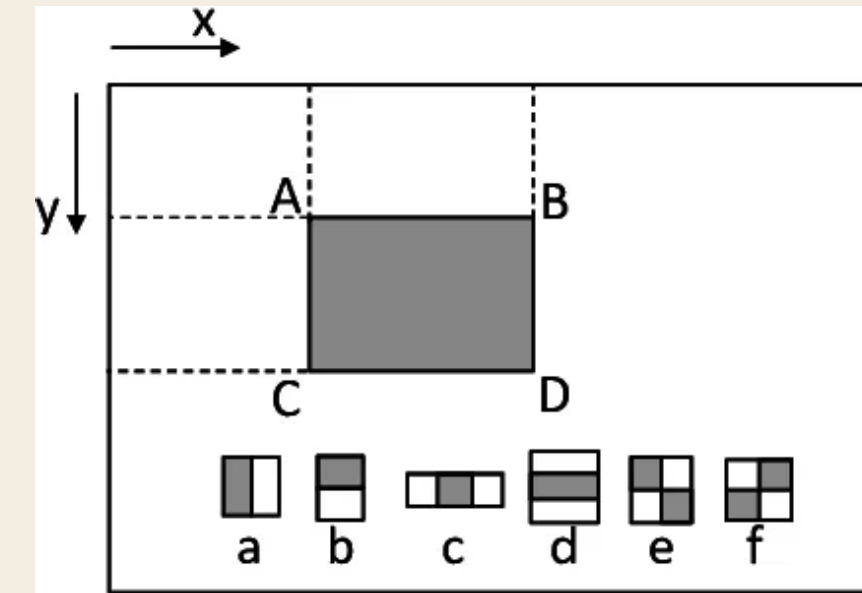
$$\begin{bmatrix} 1 & 1+2 = 3 \\ 1+3 = 4 & 1+2+3+4 = 10 \end{bmatrix}$$

Hasil:

$$\begin{bmatrix} 1 & 3 \\ 4 & 10 \end{bmatrix}$$

Cara Kerja:

Setiap nilai baru merupakan hasil penjumlahan seluruh piksel sebelumnya. Dengan metode ini sistem tidak perlu menghitung ulang seluruh area gambar sehingga proses deteksi menjadi lebih cepat.



Menggunakan trik matematika agar komputer tidak lemot saat memproses foto resolusi tinggi.

- Masalah: Menghitung ribuan piksel satu per satu itu berat.
- Solusi: Gambar diubah menjadi Integral Image, di mana setiap titik adalah hasil penjumlahan semua piksel di atas dan di kirinya.
- Keuntungan: Untuk menghitung area di dalam kotak (seperti area D pada gambar), komputer hanya butuh 4 titik sudut saja ($\text{Sum} = D - B - C + A$). Ini alasan kenapa sistem dapat mendeteksi objek secara real-time.

PROSES ADABOOST

AdaBoost digunakan untuk memilih fitur yang paling efektif dalam proses deteksi objek.

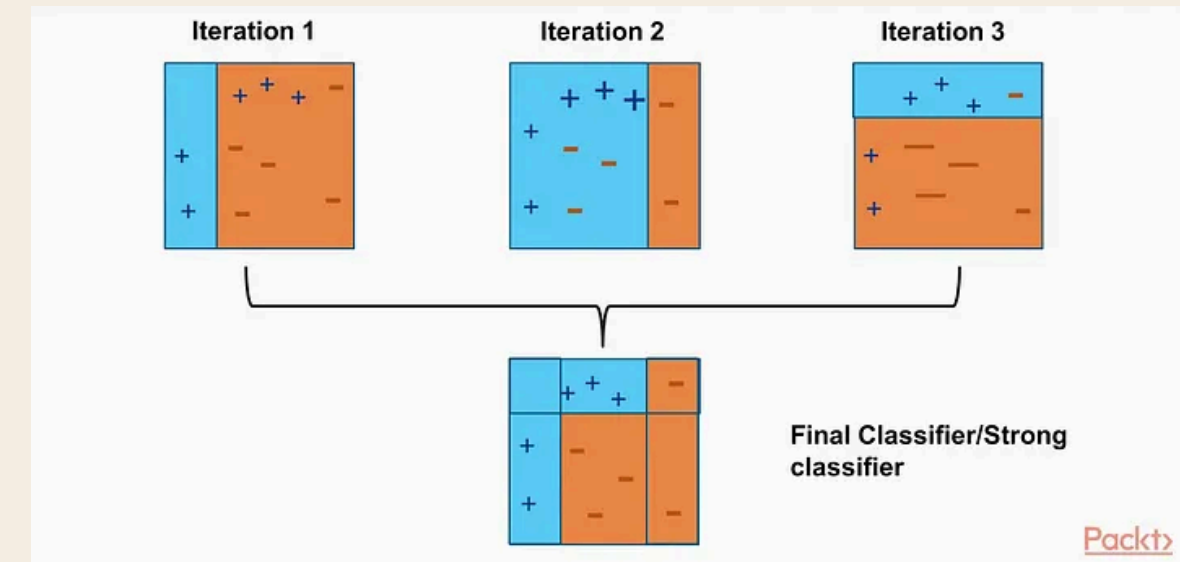
Contoh:

- Feature 1 : 68%
- Feature 2 : 91%
- Feature 3 : 74%

Sistem akan lebih memprioritaskan feature dengan nilai akurasi tertinggi untuk meningkatkan hasil deteksi.

Cara Kerja:

- sistem mencoba banyak feature
- feature terbaik dipilih
- feature lemah akan dikurangi
- proses deteksi menjadi lebih optimal



Proses pemilihan fitur yang paling akurat dari ribuan kemungkinan yang ada.

- Masalah: Dalam sebuah foto, ada ribuan pola kotak (Haar Features). Kalau komputer menghitung semuanya, prosesnya bakal jadi sangat lambat.
- Solusi (AdaBoost): Algoritma ini bertugas menyeleksi dan hanya mengambil fitur yang paling relevan untuk mengenali plat nomor atau wajah.
- Cara Kerja (Sesuai Gambar): * Sistem menggabungkan banyak "pembelajar lemah" (weak learners) menjadi satu "pembelajar kuat" (strong learner).
 - Fitur yang paling menonjol (seperti kontras antara angka plat dan latar belakangnya) akan diberi bobot lebih tinggi.
- Hasilnya: Komputer Jey tidak perlu memeriksa ribuan hal yang tidak penting. Cukup fokus pada beberapa fitur kunci yang sudah dipilih oleh AdaBoost, sehingga deteksi jadi jauh lebih cepat.

PROSES CASCADE CLASSIFIER

Cascade Classifier bekerja dengan melakukan penyaringan area gambar secara bertahap.

Contoh proses:

1000 area gambar → 350 area diproses → 80 area terdeteksi → Objek valid ditemukan

Cara Kerja:

- area yang tidak memiliki pola objek langsung dibuang
- sistem hanya melanjutkan area yang memiliki kemungkinan objek
- proses deteksi menjadi lebih cepat dan ringan

Metode ini membantu Haar Cascade melakukan deteksi secara efisien pada citra.

CARA KERJA SISTEM

Single Face Detection

Sistem membaca citra dalam bentuk grayscale kemudian melakukan scanning pada seluruh area gambar menggunakan Haar Cascade. Jika pola wajah ditemukan, sistem akan memberikan bounding box pada area wajah yang terdeteksi.

Face Blur

Setelah wajah berhasil dideteksi, sistem mengambil area wajah berdasarkan koordinat hasil deteksi lalu memberikan efek Gaussian Blur pada area tersebut agar wajah terlihat samar.

Plate Blur dan Crop

Area plat yang berhasil dideteksi akan dipotong menggunakan koordinat hasil deteksi. Setelah itu sistem dapat mengaburkan area plat atau mengambil potongan plat untuk proses OCR.

Multi Face Detection

Sistem melakukan proses scanning pada seluruh citra untuk mencari lebih dari satu pola wajah. Setiap wajah yang berhasil dikenali akan ditandai menggunakan bounding box secara otomatis.

Plate Detection

Sistem membaca citra kendaraan kemudian melakukan scanning menggunakan classifier plat nomor kendaraan. Jika area plat ditemukan, sistem akan memberikan tanda kotak pada posisi plat.

OCR Plate Recognition

Hasil crop plat diproses menggunakan EasyOCR untuk membaca karakter pada plat kendaraan. Sistem kemudian menampilkan teks hasil pembacaan beserta nilai probabilitasnya.

DATASET YANG DIGUNAKAN

Dataset menggunakan citra pribadi yang dibagi menjadi:

Dataset	Jumlah
Single Face	3
Multi Face	2
Plate Detection	4

Link Dataset: <https://drive.google.com/drive/folders/1APogt91MLYfPFxgvskFEnloZt7o8u7ir?usp=sharing>

ANALISIS KODE

Instalasi Library

```
!pip install opencv-python  
!pip install easyocr
```

- !pip install opencv-python: Perintah untuk mengunduh dan menginstal library OpenCV yang digunakan untuk pengolahan gambar dan video.
- !pip install easyocr: Perintah untuk mengunduh dan menginstal library EasyOCR yang berfungsi untuk mendeteksi serta membaca teks di dalam gambar (OCR).

ANALISIS KODE

Import Library Utama, Koneksi Google Drive

```
import cv2
import matplotlib.pyplot as plt
import numpy as np
import os
```

```
from google.colab import drive
drive.mount('/content/drive')
path = "/content/drive/MyDrive/GoogleColab_ComputerVision/Tugas3/Dataset"
```

```
Mounted at /content/drive
```

- `import cv2`: Library OpenCV untuk fungsi pengolahan citra digital.
- `import matplotlib.pyplot as plt`: Modul untuk visualisasi data dan menampilkan gambar.
- `import numpy as np`: Digunakan untuk operasi matriks dan perhitungan numerik pada array gambar.
- `import os`: Digunakan untuk manajemen file dan navigasi direktori folder dataset.
- `from google.colab import drive`: Memanggil modul sistem Drive.
- `drive.mount('/content/drive')`: Menghubungkan direktori Google Drive ke sistem Colab.
- `path = "..."`: Mendefinisikan lokasi spesifik folder dataset yang akan diproses.

ANALISIS KODE

Visualisasi Dataset

```
folder = os.path.join(path, "single_face")
files = os.listdir(folder)
plt.figure(figsize=(15,5))
for i, file in enumerate(files):
    img_path = os.path.join(folder, file)
    img = cv2.imread(img_path, 0)

    plt.subplot(1, len(files), i+1)
    plt.imshow(img, cmap='gray')
    plt.title(file)
    plt.axis('off')
plt.show()
```

single1.jpeg



single2.jpeg



single3.jpeg



- `folder = os.path.join(path, "single_face")`: Menentukan target sub-folder yang berisi gambar wajah tunggal.
- `files = os.listdir(folder)`: Mengambil seluruh daftar nama file gambar yang tersedia di dalam folder tersebut.
- `plt.figure(figsize=(15,5))`: Menyiapkan kanvas tampilan dengan ukuran lebar 15 dan tinggi 5 inci.
- `for i, file in enumerate(files)::` Melakukan perulangan untuk memproses setiap gambar berdasarkan urutan indeksnya.
- `img_path = os.path.join(folder, file)`: Menggabungkan nama file dengan direktori folder untuk akses path lengkap.
- `cv2.imread(img_path, 0)`: Membaca data citra dan langsung mengubahnya ke mode hitam-putih (grayscale).
- `plt.subplot(1, len(files), i+1)`: Mengatur posisi gambar agar tersusun rapi secara horizontal dalam satu baris.
- `plt.imshow(img, cmap='gray')`: Menampilkan citra ke layar menggunakan skema warna abu-abu.
- `plt.title(file)`: Menampilkan nama file di atas masing-masing gambar sebagai label identitas.
- `plt.axis('off')`: Menonaktifkan garis koordinat (angka sumbu) agar tampilan visual lebih bersih.
- `plt.show()`: Instruksi akhir untuk merender dan memunculkan seluruh output visualisasi.

ANALISIS KODE

Deteksi Single Face

```
haarcascades = cv2.data.haarcascades + 'haarcascade_frontalface_default.xml'  
face_cascade = cv2.CascadeClassifier(haarcascades)
```

```
deteksi_wajah = face_cascade.detectMultiScale(  
    img,  
    scaleFactor=1.1,  
    minNeighbors=5  
)
```

- `haarcascades = ... + 'haarcascade_frontalface_default.xml':` Menentukan lokasi file model pre-trained Haar Cascade yang khusus digunakan untuk mendeteksi wajah tampak depan.
- `face_cascade = cv2.CascadeClassifier(haarcascades):` Memuat model classifier ke dalam OpenCV agar siap digunakan untuk proses deteksi.
- `deteksi_wajah = face_cascade.detectMultiScale(...):` Fungsi utama untuk mendeteksi objek wajah dalam gambar, dengan parameter pendukung:
- `img:` Gambar sumber yang akan diproses (biasanya dalam format grayscale).
- `scaleFactor=1.1:` Menentukan seberapa besar ukuran gambar dikurangi pada setiap skala gambar untuk mendeteksi wajah dengan ukuran berbeda.
- `minNeighbors=5:` Menentukan berapa banyak tetangga (kandidat deteksi) yang harus dipertahankan agar sebuah objek divalidasi sebagai wajah (mengurangi false positive).

ANALISIS KODE

Deteksi Single Face

```
hasil_deteksi = []
for file in files:
    if file.endswith(('.jpg', '.png', '.jpeg')):
        img_path = os.path.join(folder, file)
        img = cv2.imread(img_path, 0)
        deteksi_wajah = face_cascade.detectMultiScale(img)
        for (x,y,w,h) in deteksi_wajah:
            cv2.rectangle(img,
                          (x,y),
                          (x+w, y+h),
                          (0,0,0),
                          5)
        hasil_deteksi.append((file, img))
```

- `hasil_deteksi = []`: Menyiapkan list kosong untuk menampung hasil akhir gambar yang telah diproses.
- `if file.endswith(...)`: Melakukan filter untuk memastikan hanya file dengan format gambar (jpg, png, jpeg) yang akan diproses oleh sistem.
- `img = cv2.imread(img_path, 0)`: Membaca gambar dalam mode grayscale untuk mengoptimalkan kinerja algoritma deteksi.
- `deteksi_wajah = face_cascade.detectMultiScale(img)`: Menjalankan fungsi deteksi untuk mencari koordinat wajah (x, y, lebar, tinggi) pada gambar.
- `for (x,y,w,h) in deteksi_wajah:`: Melakukan perulangan untuk setiap wajah yang ditemukan dalam satu gambar.
- `cv2.rectangle(...)`: Menggambar kotak pembatas (bounding box) pada area wajah yang terdeteksi dengan parameter:
 - (x, y) & (x+w, y+h): Koordinat titik pojok kiri atas dan kanan bawah kotak.
 - (0, 0, 0): Warna garis kotak (hitam).
 - 5: Ketebalan garis kotak sebesar 5 piksel.
- `hasil_deteksi.append((file, img))`: Menyimpan pasangan nama file dan data gambar hasil deteksi ke dalam list `hasil_deteksi`.

ANALISIS KODE

Visualisasi Deteksi Single Face

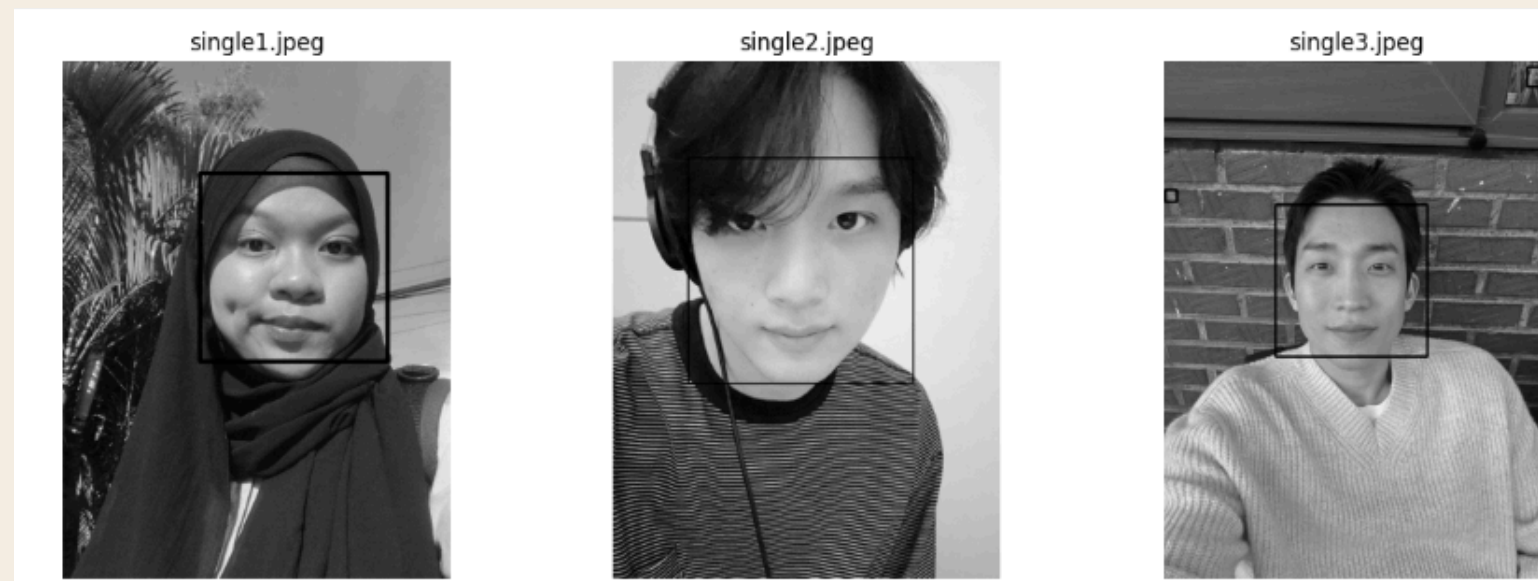
```
plt.figure(figsize=(15,5))

for i, (file, img) in enumerate(hasil_deteksi):
    plt.subplot(1, len(hasil_deteksi), i+1)
    plt.imshow(img, cmap='gray')
    plt.title(file)
    plt.axis('off')
plt.show()
```

- `plt.figure(figsize=(15,5))`: Mengatur ukuran kanvas visualisasi agar cukup lebar untuk menampilkan beberapa gambar sekaligus.
- `for i, (file, img) in enumerate(hasil_deteksi):`: Melakukan iterasi untuk mengambil data nama file dan gambar yang sudah diproses dari list `hasil_deteksi`.
- `plt.subplot(1, len(hasil_deteksi), i+1)`: Membuat tata letak grid satu baris dengan jumlah kolom sebanyak total gambar hasil deteksi.
- `plt.imshow(img, cmap='gray')`: Menampilkan gambar ke layar dalam skema warna abu-abu.
- `plt.title(file)`: Memberikan judul di atas gambar sesuai dengan nama file aslinya.
- `plt.axis('off')`: Menghilangkan indikator sumbu (angka koordinat) agar tampilan visual fokus pada objek wajah.
- `plt.show()`: Perintah untuk merender dan memunculkan seluruh hasil visualisasi secara bersamaan.

TAMPILAN HASIL

Visualisasi Deteksi Single Face



Berdasarkan hasil visualisasi, sistem berhasil mendeteksi seluruh wajah pada ketiga citra dengan cukup baik yang ditandai dengan munculnya bounding box pada area wajah. Penggunaan metode Haar Cascade Classifier dan mode grayscale membantu proses deteksi tetap berjalan optimal meskipun terdapat perbedaan pada setiap gambar, seperti penggunaan hijab, headphone, maupun background yang berbeda. Hasil tersebut menunjukkan bahwa sistem mampu mengenali wajah tunggal (single face) dengan posisi wajah menghadap ke depan secara akurat.

ANALISIS KODE

Deteksi Multi Face

```

folder = os.path.join(path, "multi_face")
files = os.listdir(folder)
face = cv2.data.harcascades + 'haarcascade_frontalface_default.xml'
face_cascade = cv2.CascadeClassifier(face)
plt.figure(figsize=(15,10))
for i, file in enumerate(files):
    if file.endswith(('.jpg', '.png', '.jpeg')):
        img_path = os.path.join(folder, file)
        img = cv2.imread(img_path, 0)
        deteksi_wajah = face_cascade.detectMultiScale(
            img,
            minNeighbors=4,
            scaleFactor=1.05
        )
        for (x, y, w, h) in deteksi_wajah:
            cv2.rectangle(img,
                           (x,y),
                           (x+w, y+h),
                           (255),
                           3)

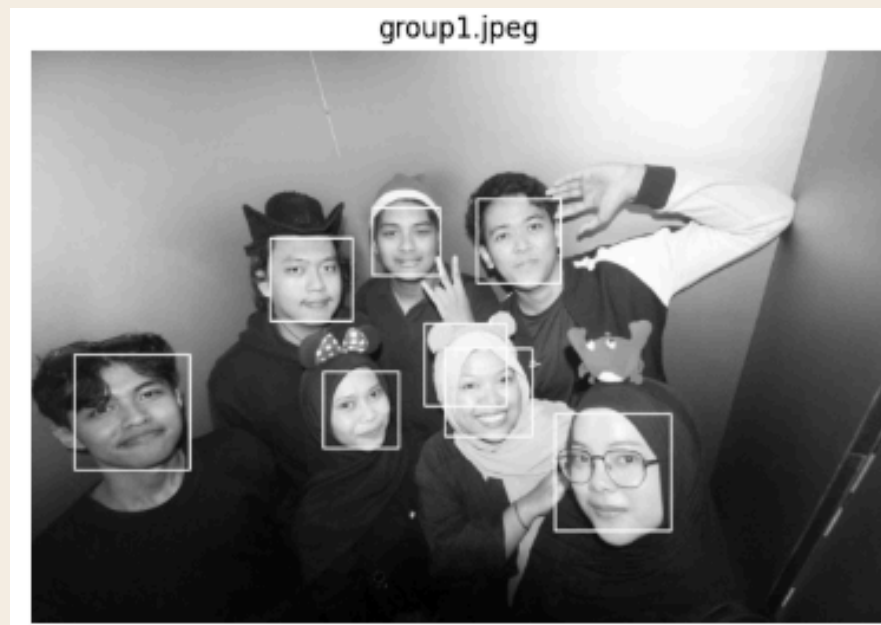
        plt.subplot(1, len(files), i+1)
        plt.imshow(img, cmap='gray')
        plt.title(file)
        plt.axis('off')
plt.show()

```

- `folder = os.path.join(path, "multi_face")`: Mengatur lokasi folder tujuan dengan menggabungkan path utama dan sub-folder bernama "multi_face".
- `files = os.listdir(folder)`: Membuat daftar (list) yang berisi seluruh nama file yang ada di dalam folder tersebut.
- `face = cv2.data.harcascades + 'haarcascade_frontalface_default.xml'`: Mendefinisikan path menuju file konfigurasi XML (model pintar) untuk mendeteksi wajah tampak depan.
- `face_cascade = cv2.CascadeClassifier(face)`: Memuat model detektor ke dalam variabel `face_cascade` agar siap digunakan.
- `plt.figure(figsize=(15,10))`: Menyiapkan area visualisasi dengan ukuran cukup besar agar gambar yang berjajar tidak terlihat sempit.
- `for i, file in enumerate(files)::` Melakukan perulangan untuk setiap file sambil mencatat urutannya (indeks i) untuk keperluan posisi tampilan.
- `if file.endswith(('.jpg', '.png', '.jpeg'))::` Logika filter untuk memastikan program hanya memproses file gambar dan mengabaikan file lain.
- `img_path = os.path.join(folder, file)`: Membuat alamat lengkap (path) untuk setiap file gambar yang akan dibaca.
- `img = cv2.imread(img_path, 0)`: Membaca gambar dan langsung mengubahnya menjadi Grayscale (hitam-putih) agar proses deteksi lebih cepat dan ringan.
- `deteksi_wajah = face_cascade.detectMultiScale(...)`: Perintah inti untuk mencari koordinat wajah dengan pengaturan
- `minNeighbors=4`: Membutuhkan minimal 4 kecocokan agar area tersebut resmi disebut "wajah".
- `scaleFactor=1.05`: Memperkecil gambar sebesar 5% pada setiap tahap pencarian untuk mendeteksi wajah dengan berbagai ukuran.
- `for (x, y, w, h) in deteksi_wajah::` Perulangan jika ditemukan lebih dari satu wajah dalam satu gambar.
- `cv2.rectangle(...)`: Menggambar kotak pembatas di sekeliling wajah:
- `(x, y) & (x+w, y+h)`: Titik koordinat kotak.
- `(255)`: Warna kotak putih (dalam mode grayscale).
- `3`: Ketebalan garis kotak.
- `plt.subplot(1, len(files), i+1)`: Mengatur posisi gambar dalam grid (1 baris, jumlah kolom sesuai jumlah file, urutan ke-i).
- `plt.imshow(img, cmap='gray')`: Menaruh data gambar ke dalam plot dengan peta warna abu-abu.
- `plt.title(file)`: Memberikan judul di atas gambar menggunakan nama file aslinya.
- `plt.axis('off')`: Menghilangkan angka-angka koordinat pada pinggir gambar agar rapi.
- `plt.show()`: Instruksi terakhir untuk menampilkan seluruh hasil deteksi ke layar.

TAMPILAN HASIL

Visualisasi Deteksi Multi Face



Berdasarkan output visualisasi pada gambar group1.jpeg dan group2.jpeg, algoritma Haar Cascade Classifier menunjukkan kemampuan yang cukup baik dalam mendeteksi banyak wajah sekaligus dalam satu frame (multi-face detection). Sistem berhasil mengidentifikasi sebagian besar wajah subjek meskipun terdapat variasi posisi, ekspresi, maupun penggunaan aksesoris seperti kacamata dan penutup kepala. Namun, pada group2.jpeg, terlihat adanya false positive atau tumpang tindih kotak deteksi pada area yang sama, yang mengindikasikan bahwa nilai scaleFactor yang rendah (1.05) membuat model sangat sensitif dalam memindai fitur wajah namun rentan terhadap deteksi ganda jika fitur wajah subjek terlalu berdekatan.

ANALISIS KODE

Deteksi Multi Face dengan Parameter Berbeda

```

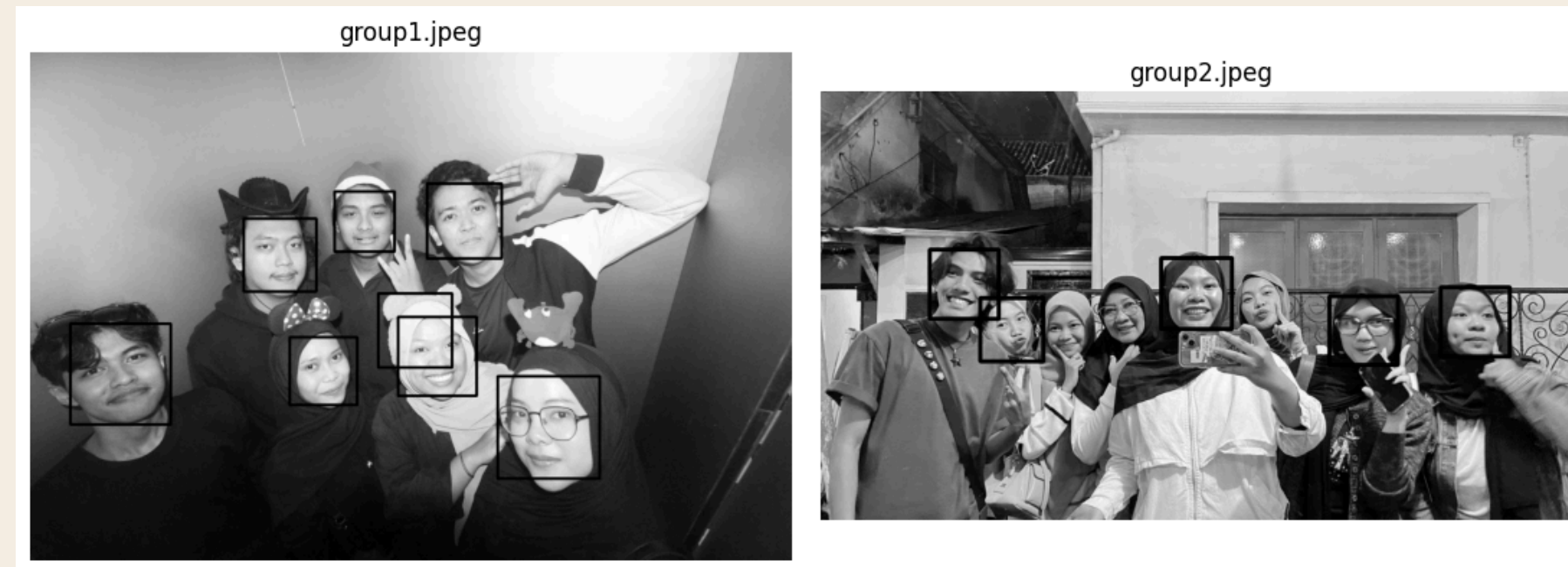
folder = os.path.join(path, "multi_face")
files = os.listdir(folder)
face = cv2.data.harcascades + 'haarcascade_frontalface_default.xml'
face_cascade = cv2.CascadeClassifier(face)
plt.figure(figsize=(15,10))
for i, file in enumerate(files):
    if file.endswith(('.jpg', '.png', '.jpeg')):
        img_path = os.path.join(folder, file)
        img = cv2.imread(img_path, 0)
        deteksi_wajah = face_cascade.detectMultiScale(
            img,
            scaleFactor=1.04,
            minNeighbors=8,
            minSize=(100,100)
        )
        for (x,y,w,h) in deteksi_wajah:
            cv2.rectangle(img,
                           (x,y),
                           (x+w, y+h),
                           (0,0,0),
                           5)
        plt.subplot(1, len(files), i+1)
        plt.imshow(img, cmap='gray')
        plt.title(file)
        plt.axis('off')
plt.show()

```

- `folder = os.path.join(path, "multi_face")`: Mengatur alamat folder tujuan ke sub-folder bernama "multi_face".
- `files = os.listdir(folder)`: Mengambil semua daftar nama file gambar yang ada di dalam folder tersebut.
- `face = cv2.data.harcascades + '...'`: Mengambil lokasi file XML cerdas bawaan OpenCV untuk mendeteksi wajah tampak depan.
- `face_cascade = cv2.CascadeClassifier(face)`: Mengaktifkan mesin pendeteksi wajah berdasarkan data XML tadi.
- `for i, file in enumerate(files)::` Melakukan perulangan untuk memproses gambar satu per satu sekaligus mencatat urutannya (i).
- `img = cv2.imread(img_path, 0)`: Membaca gambar dan langsung mengubah warnanya menjadi abu-abu (grayscale) agar deteksi lebih ringan.
- `scaleFactor=1.04`: Mengecilkan ukuran gambar sebesar 4% di setiap tahap pengecekan untuk menemukan wajah dari berbagai jarak.
- `minNeighbors=8`: Syarat bahwa sebuah area harus terdeteksi 8 kali oleh sistem untuk dinyatakan sebagai wajah asli (mengurangi salah deteksi).
- `minSize=(100,100)`: Menentukan batas ukuran wajah terkecil yang boleh dideteksi adalah 100x100 piksel.
- `cv2.rectangle(..., 5)`: Menggambar kotak di sekeliling wajah yang ditemukan dengan ketebalan garis 5 piksel.
- `plt.subplot(1, len(files), i+1)`: Mengatur tata letak agar semua gambar hasil deteksi tampil berjajar dalam satu baris.
- `plt.imshow(img, cmap='gray')`: Menampilkan gambar ke dalam frame dengan skema warna abu-abu.

TAMPILAN HASIL

Deteksi Multi Face dengan Parameter Berbeda



Berdasarkan output visualisasi pada gambar group1.jpeg dan group2.jpeg, algoritma Haar Cascade Classifier menunjukkan kemampuan yang sangat akurat dalam mengidentifikasi wajah pada area utama (foreground). Penggunaan parameter minNeighbors=8 terbukti efektif dalam menghasilkan deteksi yang bersih dan bebas dari false positive. Namun, pada group2.jpeg, terdapat 3 wajah pada area latar belakang yang tidak berhasil terdeteksi. Hal ini disebabkan oleh penerapan nilai minSize=(100,100) yang cukup tinggi; wajah subjek yang berada jauh di belakang memiliki resolusi piksel yang terlalu kecil untuk memenuhi ambang batas tersebut. Analisis ini menunjukkan bahwa model sangat stabil untuk objek jarak dekat, namun memiliki keterbatasan sensitivitas terhadap objek berukuran kecil demi menjaga ketajaman akurasi deteksi utama.

ANALISIS KODE

Deteksi & Blurring Multi Face

```

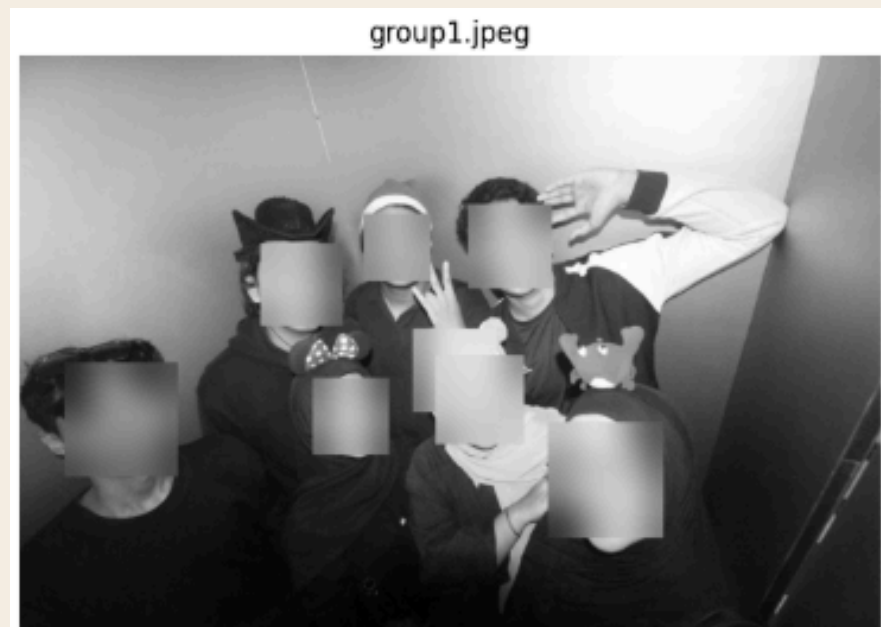
folder = os.path.join(path, "multi_face")
files = os.listdir(folder)
face = cv2.data.harcascades + 'haarcascade_frontalface_default.xml'
face_cascade = cv2.CascadeClassifier(face)
plt.figure(figsize=(15,10))
for i, file in enumerate(files):
    if file.endswith((''.jpg', '.png', '.jpeg')):
        img_path = os.path.join(folder, file)
        img = cv2.imread(img_path, 0)
        deteksi_wajah = face_cascade.detectMultiScale(
            img,
            scaleFactor=1.04,
            minNeighbors=8
        )
        for (x, y, w, h) in deteksi_wajah:
            target = img[y:y+h, x:x+w]
            blur = cv2.GaussianBlur(target, (251,251), 0)
            img[y:y+h, x:x+w] = blur
        plt.subplot(1, len(files), i+1)
        plt.imshow(img, cmap='gray')
        plt.title(file)
        plt.axis('off')
plt.show()

```

- `deteksi_wajah = face_cascade.detectMultiScale(...)`: Mencari objek wajah dengan parameter:
- `scaleFactor=1.04`: Memindai gambar dengan perubahan skala 4% untuk akurasi pada wajah yang ukurannya bervariasi.
- `minNeighbors=8`: Menggunakan nilai ambang batas yang lebih tinggi (8) untuk memastikan hanya area yang benar-benar mirip wajah yang diproses (mengurangi deteksi palsu).
- `target = img[y:y+h, x:x+w]`: Melakukan proses cropping atau pemotongan area gambar secara spesifik hanya pada koordinat wajah yang terdeteksi.
- `blur = cv2.GaussianBlur(target, (251,251), 0)`: Menerapkan teknik Gaussian Blur pada area wajah. Ukuran kernel (251,251) yang besar digunakan untuk memberikan efek sensor yang sangat kuat (sangat kabur).
- `img[y:y+h, x:x+w] = blur`: Menimpa area wajah asli pada gambar dengan area yang sudah dikaburkan.
- `plt.imshow(img, cmap='gray')`: Menampilkan hasil akhir gambar yang sudah disensor ke dalam plot.
- `plt.title(file)`: Menampilkan nama file sebagai identitas di atas hasil visualisasi.

TAMPILAN HASIL

Visualisasi Deteksi Multi Face



Berdasarkan output visualisasi pada gambar group1.jpeg dan group2.jpeg, algoritma deteksi wajah yang dikombinasikan dengan teknik Gaussian Blur telah berhasil mengidentifikasi sekaligus menyensor identitas seluruh subjek dalam gambar secara otomatis. Pada group1.jpeg, sistem mampu mendeteksi dan menutupi wajah dengan rapi meskipun posisi subjek bervariasi. Sementara pada group2.jpeg, terlihat intensitas pengaburan yang sangat efektif menggunakan kernel yang besar, sehingga informasi detail wajah benar-benar hilang untuk menjaga privasi. Meskipun terdapat beberapa area pengaburan yang saling tumpang tindih karena posisi subjek yang sangat berdekatan, fungsi utama sistem untuk melindungi identitas dalam foto berkelompok (multi-face) telah terpenuhi dengan sangat baik.

ANALISIS KODE

Persiapan Citra Plat Kendaraan

```
folder = os.path.join(path, 'plate_detection')
files = os.listdir(folder)
images = []
plt.figure(figsize=(15,10))
for i, file in enumerate(files):
    if file.endswith((''.jpg', '.png', '.jpeg')):
        car_plate = os.path.join(folder, file)
        img = cv2.imread(car_plate, 0)
        images.append((file, img))
        plt.subplot(1, len(files), i+1)
        plt.axis('off')
        plt.imshow(img, cmap='gray')
        plt.title(file)
plt.show()
```



- `folder = os.path.join(path, 'plate_detection')`: Menentukan lokasi folder spesifik bernama "plate_detection" di dalam direktori dataset.
- `files = os.listdir(folder)`: Mengambil daftar seluruh nama file yang tersimpan di dalam folder tersebut.
- `images = []`: Menyiapkan list kosong untuk menyimpan data gambar yang telah dibaca.
- `plt.figure(figsize=(15,10))`: Mengatur dimensi bingkai visualisasi agar tampilan gambar berjajar terlihat proporsional.
- `for i, file in enumerate(files)::` Melakukan perulangan untuk memproses setiap file berdasarkan urutan indeksinya.
- `if file.endswith((''.jpg', '.png', '.jpeg'))::` Melakukan filter agar sistem hanya memproses file dengan format ekstensi gambar yang valid.
- `car_plate = os.path.join(folder, file)`: Membuat alamat lengkap (path) untuk akses setiap file gambar.
- `img = cv2.imread(car_plate, 0)`: Membaca gambar menggunakan OpenCV dan langsung mengubahnya ke mode Grayscale (nilai 0).
- `images.append((file, img))`: Memasukkan pasangan nama file dan data matriks gambar ke dalam list images.
- `plt.subplot(1, len(files), i+1)`: Mengatur tata letak tampilan gambar dalam satu baris horizontal secara otomatis.
- `plt.axis('off')`: Menghilangkan indikator sumbu koordinat agar hasil visualisasi lebih bersih.
- `plt.imshow(img, cmap='gray')`: Menampilkan citra ke layar dengan skema warna abu-abu.
- `plt.title(file)`: Memberikan judul di atas setiap gambar sesuai dengan nama file aslinya.
- `plt.show()`: Instruksi akhir untuk menampilkan seluruh rangkaian proses visualisasi ke layar.

ANALISIS KODE

Deteksi Plat Kendaraan

```
hasil_deteksi = []
haarcascades = cv2.data.haarcascades + 'haarcascade_russian_plate_number.xml'
plat_cascade = cv2.CascadeClassifier(haarcascades)
for file, img in images:
    img2 = img.copy()
    print(file, img2.shape)
    deteksi_plate = plat_cascade.detectMultiScale(
        img2,
        scaleFactor=1.05,
        minNeighbors=7,
        minSize=(30,30)
    )
    hasil_deteksi.append((file, img2, deteksi_plate))
```

```
plate1.jpeg (3024, 4032)
plate4.jpeg (4032, 3024)
plate2.jpeg (2273, 1705)
plate3.jpeg (2170, 2893)
```

- `hasil_deteksi = []`: Menyiapkan list kosong untuk menyimpan hasil koordinat plat nomor yang ditemukan dari setiap gambar.
- `haarcascades = ... + 'haarcascade_russian_plate_number.xml'`: Mendefinisikan path menuju file XML Haar Cascade yang khusus dilatih untuk mendeteksi plat nomor kendaraan.
- `plat_cascade = cv2.CascadeClassifier(haarcascades)`: Memuat model detektor plat nomor ke dalam sistem OpenCV.
- `for file, img in images::` Melakukan perulangan untuk memproses setiap gambar mobil yang sudah disiapkan sebelumnya dalam list images.
- `img2 = img.copy()`: Menduplikasi gambar asli agar proses deteksi tidak merusak data gambar mentah yang asli.
- `print(file, img2.shape)`: Menampilkan nama file beserta dimensi gambar (tinggi dan lebar) untuk memastikan data terbaca dengan benar.
- `deteksi_plate = plat_cascade.detectMultiScale(...)`: Fungsi inti untuk mencari letak plat nomor dengan parameter:
- `img2`: Gambar input yang diproses.
- `scaleFactor=1.05`: Memperkecil ukuran gambar sebesar 5% pada setiap tahap pemindaian untuk mendeteksi plat dengan ukuran yang bervariasi.
- `minNeighbors=7`: Parameter untuk validasi deteksi; membutuhkan 7 tetangga kandidat yang cocok agar area tersebut dipastikan sebagai plat nomor (menghindari deteksi palsu).
- `minSize=(30,30)`: Batasan ukuran minimal objek yang dicari agar sistem tidak memproses titik-titik kecil yang bukan plat nomor.
- `hasil_deteksi.append((file, img2, deteksi_plate))`: Menyimpan nama file, data gambar, dan hasil koordinat deteksi ke dalam list untuk tahap visualisasi berikutnya.

ANALISIS KODE

Visualisasi Deteksi Plat

```
plt.figure(figsize=(15,10))
for i, (file, img2, deteksi_plate) in enumerate(hasil_deteksi):
    for(x,y,w,h) in deteksi_plate:
        cv2.rectangle(img2,
                       (x,y),
                       (x+w, y+h),
                       (255),
                       15)
    plt.subplot(1, len(hasil_deteksi), i+1)
    plt.axis('off')
    plt.imshow(img2, cmap='gray')
    plt.title(file)
plt.show()
```

- `plt.figure(figsize=(15,10))`: Menyiapkan kanvas tampilan dengan ukuran lebar 15 dan tinggi 10 inci agar gambar terlihat jelas.
- `for i, (file, img2, deteksi_plate) in enumerate(hasil_deteksi):`: Melakukan perulangan untuk mengambil nama file, data gambar, dan hasil koordinat deteksi yang sudah disimpan sebelumnya.
- `for(x,y,w,h) in deteksi_plate:`: Mengambil koordinat spesifik (titik x, y, lebar, dan tinggi) dari setiap plat nomor yang terdeteksi dalam satu gambar.
- `cv2.rectangle(...)`: Menggambar kotak penanda pada gambar dengan parameter:
 - `img2`: Gambar target.
 - `(x,y) & (x+w, y+h)`: Koordinat diagonal untuk membentuk kotak.
 - `(255)`: Warna kotak putih (karena menggunakan mode grayscale).
 - `15`: Ketebalan garis kotak (dibuat tebal agar terlihat jelas pada resolusi tinggi).
- `plt.subplot(1, len(hasil_deteksi), i+1)`: Mengatur tata letak agar gambar tampil berjajar ke samping sesuai jumlah data.
- `plt.axis('off')`: Menghilangkan angka sumbu koordinat pada pinggir gambar.
- `plt.imshow(img2, cmap='gray')`: Menampilkan gambar hasil deteksi dengan peta warna abu-abu.
- `plt.title(file)`: Memberikan judul di atas setiap gambar sesuai nama filenya.
- `plt.show()`: Perintah final untuk memunculkan semua hasil visualisasi ke layar.

TAMPIILAN HASIL

Visualisasi Deteksi Plat



Hasil pengujian menunjukkan bahwa model Haar Cascade memiliki tingkat keberhasilan sebesar 50% pada dataset ini. Deteksi berhasil dilakukan secara akurat pada plate2.jpeg dan plate4.jpeg karena posisi plat nomor yang cenderung lurus (frontal) dan memiliki kontras yang jelas terhadap bodi kendaraan. Sebaliknya, kegagalan deteksi (false negative) terjadi pada plate1.jpeg dan plate3.jpeg yang disebabkan oleh sudut kemiringan objek yang terlalu ekstrem serta jarak objek yang terlalu jauh dari kamera. Hal ini mengindikasikan bahwa model sangat bergantung pada orientasi objek dan resolusi area target; sehingga diperlukan optimasi pada parameter minNeighbors atau penggunaan teknik Image Enhancement untuk meningkatkan akurasi pada kondisi sudut pandang yang sulit.

ANALISIS KODE

Visualisasi Blur Deteksi Plat

```
plt.figure(figsize=(15,10))
for i, (file, img, deteksi_plate) in enumerate(hasil_deteksi):
    img3 = img.copy()
    for(x,y,w,h) in deteksi_plate:
        target = img3[y:y+h, x:x+w]
        target = cv2.GaussianBlur(target, (101,101), 0)
        img3[y:y+h, x:x+w] = target
    plt.subplot(1, len(hasil_deteksi), i+1)
    plt.axis('off')
    plt.imshow(img3, cmap='gray')
    plt.title(file)
plt.show()
```

- `img3 = img.copy()`: Membuat salinan gambar agar proses pengaburan tidak mengubah data gambar asli yang disimpan dalam variabel `img` pada deteksi sebelumnya.
- `target = img3[y:y+h, x:x+w]`: Melakukan cropping otomatis tepat pada area koordinat plat nomor yang sudah dideteksi sebelumnya.
- `target = cv2.GaussianBlur(target, (101,101), 0)`: Menerapkan efek Gaussian Blur pada area plat. Angka (101,101) adalah ukuran kernel; semakin besar angkanya, maka hasil sensor akan semakin kabur/tidak terbaca.
- `img3[y:y+h, x:x+w] = target`: Memasang kembali potongan gambar yang sudah dibuat kabur ke posisi aslinya di dalam gambar `img3`.
- `plt.imshow(img3, cmap='gray')`: Menampilkan hasil akhir gambar mobil dengan plat nomor yang sudah disensor.
- `plt.title(file)`: Menampilkan nama file di atas gambar hasil olahan.

TAMPIILAN HASIL

Visualisasi Blur Deteksi Plat



Dari hasil visualisasi, kita ambil kesimpulan bahwa:

- Privasi Berhasil: Pada gambar plate2 dan plate4, plat nomornya sudah tertutup kabur (blur). Angka dan hurufnya jadi tidak bisa dibaca sama sekali. Ini tandanya fungsi sensor privasi bekerja dengan sangat baik.
- Kendala Deteksi: Untuk gambar plate1 dan plate3, sensornya tidak muncul. Ini bukan karena kodenya rusak, tapi karena dari awal sistem sulit mengenali plat nomornya (posisinya terlalu miring atau jaraknya terlalu jauh).

ANALISIS KODE

Pemotongan Plat Nomor

```
cropped_plate = []
for file, img, deteksi_plate in hasil_deteksi:
    img4 = img.copy()
    print(file, img4.shape)
    for (x,y,w,h) in deteksi_plate:
        plate = img4[y:y+h, x:x+w]
        cropped_plate.append(plate)
plt.figure(figsize=(15,10))
for i, plate in enumerate(cropped_plate):
    plt.subplot(1, len(cropped_plate), i + 1)
    plt.axis('off')
    plt.imshow(plate, cmap='gray')
plt.show()
```

- `cropped_plate = []`: Menyiapkan daftar kosong untuk menyimpan potongan gambar plat nomor.
- `img4 = img.copy()`: Menyalin gambar asli supaya data aslinya tetap aman saat kita otak-atik.
- `plate = img4[y:y+h, x:x+w]`: Proses memotong gambar. Sistem hanya mengambil area di dalam koordinat (kotak) plat nomor yang sudah dideteksi.
- `cropped_plate.append(plate)`: Memasukkan hasil potongan tadi ke dalam daftar `cropped_plate`.
- `for i, plate in enumerate(cropped_plate)::` Mengulangi proses untuk menampilkan semua potongan plat nomor yang sudah dikumpulkan.
- `plt.imshow(plate, cmap='gray')`: Menampilkan potongan plat nomor dalam warna hitam-putih (grayscale).

TAMPILAN HASIL

Visualisasi Pemotongan Plat Nomor

```
plate1.jpeg (3024, 4032)  
plate4.jpeg (4032, 3024)  
plate2.jpeg (2273, 1705)  
plate3.jpeg (2170, 2893)
```



Berdasarkan hasil gambarnya, berikut poin-poin simpelnya:

- Ekstraksi Berhasil: Sistem berhasil memisahkan objek plat nomor dari gambar kendaraan secara keseluruhan.
- Fokus Objek: Hasil potongan (cropping) memudahkan kita untuk melihat detail angka dan huruf pada plat nomor tanpa terganggu oleh latar belakang gambar yang ramai.
- Walaupun hasil potongan terdapat bukan termasuk plat nomor, menandakan bahwa model dapat mendeteksi hal-hal sensitif terkait informasi pribadi.

ANALISIS KODE

OCR (membaca teks/angka dari potongan plat nomor)

```
import easyocr
reader = easyocr.Reader(['en'])
for plate in cropped_plate:
    th, plate = cv2.threshold(
        plate,
        100,
        255,
        cv2.THRESH_BINARY
    )
    plate = cv2.blur(plate, (3,3))
    result = reader.readtext(plate)
    for (bbox, text, prob) in result:
        print(f'Text: {text}, Probability: {prob}')
```

- `import easyocr`: Memasukkan pustaka/alat bernama EasyOCR ke dalam program agar kita bisa membaca teks dari gambar.
- `reader = easyocr.Reader(['en'])`: Menyiapkan mesin pembaca teks dan memberitahunya untuk fokus mengenali karakter bahasa Inggris (huruf latin).
- `for plate in cropped_plate::` Mengambil satu per satu potongan gambar plat nomor yang sudah dipotong sebelumnya untuk diproses.
- `th, plate = cv2.threshold(...)`: Mengubah gambar plat nomor menjadi hitam-putih total (biner). Huruf dibuat jadi putih dan latar belakang jadi hitam (atau sebaliknya) agar mesin tidak bingung dengan warna lain.
- `100, 255, cv2.THRESH_BINARY`: Ini adalah aturan mainnya; piksel yang terang diubah jadi putih maksimal (255), yang gelap jadi hitam total.
- `plate = cv2.blur(plate, (3,3))`: Menghaluskan sedikit gambar yang sudah hitam-putih tadi agar garis-garis hurufnya tidak terlalu tajam/pecah saat dibaca mesin.
- `result = reader.readtext(plate)`: Baris paling penting! Di sini mesin EasyOCR bekerja memindai gambar dan menebak tulisan apa saja yang ada di dalamnya.
- `for (bbox, text, prob) in result::` Mengambil hasil tebakan mesin, yaitu: lokasi kotak teks (bbox), isi tulisan (text), dan tingkat keyakinan/akurasi (prob).
- `print(f'Text: {text}, Probability: {prob}')`: Menampilkan hasil bacaan plat nomor dan seberapa yakin mesin tersebut ke layar kamu.

TAMPILAN HASIL

Hasil OCR (membaca teks/angka dari potongan plat nomor)

```
Text: B 7518 TDB , Probability: 0.29850391375295204
Text: TOYOTA, Probability: 0.9169858599966939
Text: B, Probability: 0.9993957002241274
Text: 9L38, Probability: 0.5434616804122925
Text: NV, Probability: 0.9948686925153443
Text: B 3962 HBCI, Probability: 0.7591442787599795
Text: X 6626, Probability: 0.5892301523140603
Text: 42, Probability: 0.7154256152368756
```

- Pembacaan Berhasil: Sistem berhasil membaca beberapa plat dengan akurasi sangat tinggi, contohnya teks "B 3962 WBC" dan "NV" (bagian dari plat nomor) dengan nilai probabilitas di atas 0.75 - 0.99.
- Deteksi Tulisan Lain: Karena pada tahap sebelumnya sistem sempat salah memotong bagian mobil (bukan plat), EasyOCR juga ikut membaca tulisan "TOYOTA" dengan akurasi 0.91.
- Karakter yang Kurang Jelas: Beberapa teks seperti "B 7518 TDB" memiliki nilai probabilitas rendah (0.29) karena kualitas gambar potongan plat yang mungkin pecah atau pencahayaannya kurang rata saat diproses.

ANALISA SISTEM

1. Kecepatan & Efisiensi

- Pemrosesan: Penggunaan mode Grayscale dan Thresholding membuat beban kerja komputer lebih ringan dan proses deteksi jadi lebih cepat.
- Kendala Perangkat: Ada peringatan "CUDA/MPS not available", artinya sistem berjalan menggunakan CPU. Jika menggunakan GPU, proses pembacaan teks (OCR) bisa jauh lebih cepat.

2. Akurasi Deteksi (Haar Cascade)

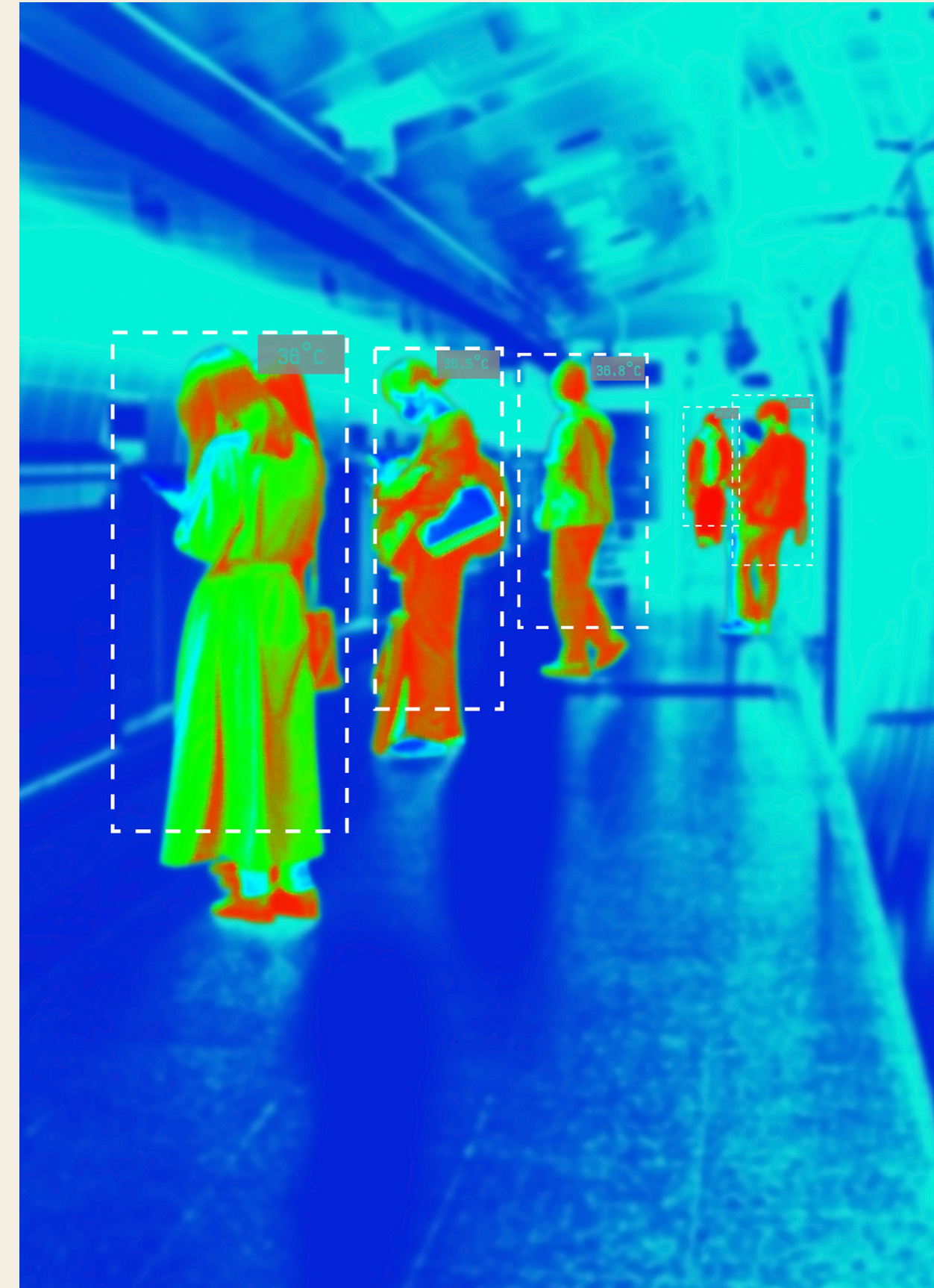
- Tingkat Keberhasilan: Berhasil pada gambar dengan posisi tegak lurus (frontal) dan kontras tinggi.
- Kelemahan: Masih kesulitan mendeteksi objek yang terlalu miring (skewed) atau terlalu jauh dari kamera karena keterbatasan fitur statis pada filter Haar.

3. Kualitas Sensor & Pembacaan (EasyOCR)

- Privasi: Fungsi Gaussian Blur sangat efektif menyembunyikan identitas kendaraan pada objek yang terdeteksi.
- Kualitas OCR: Sangat akurat (mencapai 99%) pada teks yang jelas, namun terkadang terjadi False Positive (salah mengira logo mobil "TOYOTA" sebagai plat nomor).

KESIMPULAN

Sistem ini berhasil menjalankan alur otomatis mulai dari mendeteksi, menyensor, hingga membaca teks plat nomor dengan tingkat akurasi mencapai 99% pada kondisi objek yang terlihat jelas dan lurus. Meskipun sangat efektif untuk perlindungan privasi melalui fitur blurring otomatis, sistem masih memiliki kendala dalam mengenali plat yang posisinya terlalu miring atau terlalu jauh, serta terkadang salah mengenali logo merk mobil sebagai plat nomor. Secara keseluruhan, performa sistem sudah sangat baik untuk penggunaan standar, namun perlu pengembangan lebih lanjut pada ketajaman deteksi sudut agar hasil pembacaan teks menjadi lebih stabil di berbagai kondisi posisi kendaraan.



THANK YOU

Thanks For Attention

 eoliasuwandi@gmail.com

 Link GitHub: https://github.com/seal165/KomputerVision_FaceRecognition

 LinkedIn: <https://www.linkedin.com/in/eolia-shalbillah>

 Dataset: <https://drive.google.com/drive/folders/1APoqt91MLYfPFxgvskFEnloZt7o8u7ir?usp=sharing>

Presented by Eolia Shalbillah - 2318039